

0.33.1. Écrire soi-même sa fonction — de base, ailleurs

La fonction du 0.33, appelée pour le B sur la ligne 8.

1. Ce qu'il faut comprendre

C'est le 0.33, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** dans les deux appels.

La fonction ne change pas : c'est tout l'intérêt d'une fonction. On lui donne d'autres renseignements (la ligne, la tuile), elle fait le même travail ailleurs.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.33.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1079 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      52 o  ← la console, à l'allumage — la mise en route de la console
mon_deplace_x() 156 o  ← main() — ta fonction
main()          49 o  ← démarrage — ta fonction
VBlank          8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o  ← image() — attend l'image suivante
AttendreVBlank  16 o  ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o  ← EffacerCases — attend de pouvoir écrire une case
AdresseCase     10 o  ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o  ← effacer() — efface des cases
CopierTuiles    18 o  ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o  ← démarrage — vide les deux cartes de l'écran
Tuiles          704 o  ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()        × 2    26 o sur place, dans mon_deplace_x() + AdresseCase (10 o)
effacer()      × 1    15 o sur place, dans mon_deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()        × 2     6 o sur place, dans mon_deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
mon_deplace_x() × 2    38 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>          // pose une tuile sur une case du fond
#include <effacer>         // efface des cases, ou tout le fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;           // la colonne de la lettre

// Notre fonction, la même qu'au 0.33 : elle fait avancer (ou reculer) une tuile
// de « pas » cases sur une ligne, un quart de seconde par case, puis REND la
// colonne d'arrivée.
//  uint8_t (devant le nom) = ce qu'elle REND : un octet, la colonne d'arrivée.
//  (colonne, ligne, tuile, pas) = ce qu'elle reçoit : des COPIES des valeurs.
```

3. UNE FONCTION — MON_DEPLACE_X()

```
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;          // -5 est rangé 251 : au-delà de 127, on recule
    if (recule) pas = 0 - pas;           // 251 redevient 5 : le nombre de cases
    poser(colonne, ligne, tuile);        // la lettre à sa place de départ
    while (pas > 0) {                    // tant qu'il reste des pas
        for (uint8_t i = 0; i < 15; i++) image(); // un quart de seconde (15 images)
        if (recule && colonne == 0) break;      // bord gauche : on sort de la boucle
        if (!recule && colonne == 19) break;     // bord droit : on sort de la boucle
        effacer(colonne, ligne, 1);           // 1. efface l'ancienne place (1 case)
        if (recule) colonne--;                // 2. une colonne plus à gauche...
        else colonne++;                       // ...ou plus à droite
        poser(colonne, ligne, tuile);         // 3. la lettre à sa nouvelle place
        pas--;                                // un pas de moins à faire
    }
    return colonne;                        // REND la colonne d'arrivée, et termine
}
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                          // Le programme commence ici.
    // Le changement : le B (ALPHABET[1]), sur la ligne 8.
    // « x = » range la colonne rendue par la fonction.
    x = mon_deplace_x(x, 8, ALPHABET[1], 5); // le B, ligne 8 : x vaut 5
    x = mon_deplace_x(x, 8, ALPHABET[1], -5); // retour : x vaut 0

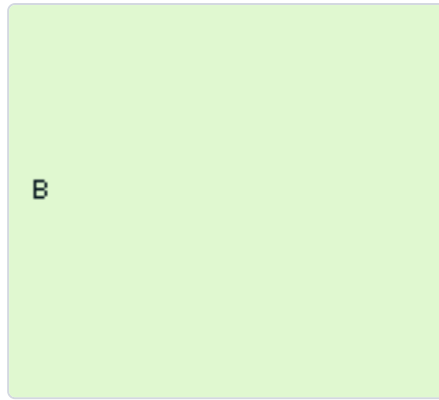
    while (true) {                     // La boucle de jeu, pour toujours :
        image();                       // attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

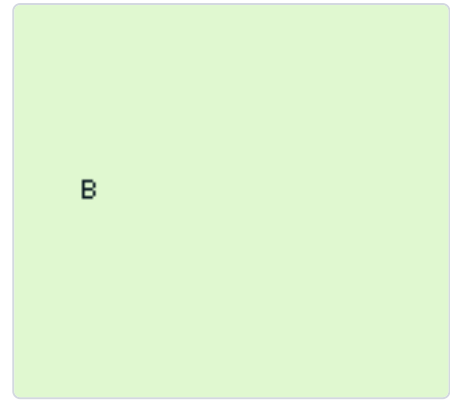
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



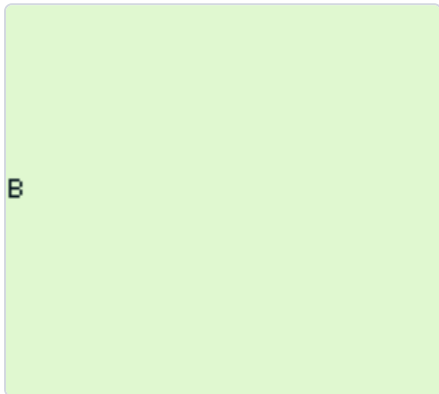
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui va et revient sur la ligne 8, grâce à notre fonction.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le B passe par (5, 8)
- ✓ le B finit en (0, 8) (0,8)

Ensuite : **0.33.2. Écrire soi-même sa fonction — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++